

Leveraging Deep Reinforcement Learning for Water Distribution Systems with Large Action Spaces and Uncertainties: DRL-EPANET for Pressure Control

Anas Belfadil¹; David Modesto, Ph.D.²; Jordi Meseguer, Ph.D.³; Bernat Joseph-Duran, Ph.D.⁴; David Saporta⁵; and Jose Antonio Martin Hernandez, Ph.D.⁶

Abstract: Deep reinforcement learning (DRL) has undergone a revolution in recent years, enabling researchers to tackle a variety of previously inaccessible sequential decision problems. However, its application to the control of water distribution systems (WDS) remains limited. This research demonstrates the successful application of DRL for pressure control in WDS by simulating an environment using EPANET version 2.2, a popular open-source hydraulic simulator. We highlight the ability of DRL-EPANET to handle large action spaces, with more than 1 million possible actions in each time step, and its capacity to deal with uncertainties such as random pipe breaks. We employ the Branching Dueling Q-Network (BDQ) algorithm, which can learn in this context, and enhance it with an algorithmic modification called BDQ with fixed actions (BDQF) that achieves better rewards, especially when manipulated actions are sparse. The proposed methodology was validated using the hydraulic models of 10 real WDS, one of which integrated transmission and distribution systems operated by Hidralia, and the rest of which were operated by Aigües de Barcelona. DOI: 10.1061/JWRMD5.WRENG-6108. © 2023 American Society of Civil Engineers.

Practical Applications: This research presents the DRL-EPANET framework, which combines deep reinforcement learning and EPANET to optimize water distribution systems. Although the focus of this paper is on pressure control, the approach is highly versatile and can be applied to various sequential decision-making problems within WDS, such as pump optimization, energy management, and water quality control. DRL-EPANET was tested and proven effective on 10 real-world WDS, resulting in as much as 26% improvement in mean pressure compared with the reference solutions. The framework offers real-time control solutions, enabling water utility operators to react quickly to changes in the network. Additionally, it is capable of handling stochastic scenarios, such as random pipe bursts, demand uncertainty, contamination, and component failures, making it a valuable tool for managing complex and unpredictable situations. This method can be developed more with the use of model-based deep reinforcement learning for enhanced sample efficiency, graph neural networks for better representation, and the quantification of agent action uncertainty for improved decision-making in uncharted situations. Overall, DRL-EPANET has the potential to revolutionize the management and operation of water distribution systems, leading to more-efficient use of resources and improved service for consumers.

¹Ph.D. Candidate, Artificial Intelligence, Dept. of Computer Science, Universitat Politècnica de Catalunya, Jordi Girona, 31, Barcelona 08034, Spain (corresponding author). ORCID: <https://orcid.org/0000-0002-9391-1350>. Email: anas.belfadil@upc.edu

²Established Researcher, Dept. of Computer Applications in Science and Engineering, Barcelona Supercomputing Center—Centro Nacional de Supercomputación, Plaça Eusebi Güell 1-3, Barcelona 08034, Spain. Email: david.modesto@bsc.es

³Project Manager/Researcher, Critical Infrastructure Management and Resilience Area, CETaqua, Water Technology Centre, Ctra. d'Esplugues 75, Cornellà del Llobregat, Barcelona 08940, Spain. ORCID: <https://orcid.org/0000-0002-0488-7556>. Email: jordi.meseguer@cetaqua.com

⁴Project Manager/Researcher, Critical Infrastructure Management and Resilience Area, CETaqua, Water Technology Centre, Ctra. d'Esplugues 75, Cornellà del Llobregat, Barcelona 08940, Spain. Email: bjoseph@cetaqua.com

⁵Engineer, Aigües de Barcelona, Dept. of Digitalisation and Operational Excellence, General Batet 1-7, Barcelona 08028, Spain. Email: david.saporta@aiguesdebarcelona.cat

⁶Technical Advisor, Advanced Mathematics, Repsol Technology Lab, P.º de Extremadura, Km 18, Móstoles, Madrid 28935, Spain. Email: ja.martin.h@repsol.com

Note. This manuscript was submitted on January 2, 2023; approved on August 29, 2023; published online on November 16, 2023. Discussion period open until April 16, 2024; separate discussions must be submitted for individual papers. This paper is part of the *Journal of Water Resources Planning and Management*, © ASCE, ISSN 0733-9496.

Introduction

Water is a limited resource with an increasing number of users. The global population has increased by almost 1.5 billion people in the last 20 years, increasing the demand for clean water. Furthermore, overexploitation of water resources has been exacerbated by urbanization, climate change, and drought. As a result, municipalities, water utility firms, and society in general must embrace more-sustainable water management techniques.

Complex and expanding water networks make it difficult to achieve satisfactory, cost-effective operations. Consequently, researchers have developed novel deterministic and stochastic (heuristic) optimization techniques (Savić et al. 2018).

Among the deterministic methods that have been developed, we have: (1) linear programming (LP), which can find optimal solutions but only works for a continuous problem with a linear objective function subject to linear constraints (Schaake and Lai 1969); (2) dynamic Programming (DP), that is suitable for multistage optimization problems, and mostly used for pump scheduling. However, it suffers from the so-called curse of dimensionality, which limits to some extent its application to large WDS (Yakowitz 1982); and (3) nonlinear programming (NLP), works with continuous spaces but is limited in the number of variables

and constraints, and thus it can only manage WDS of limited size (Deuerlein et al. 2009).

Due to the nonlinearity and discreteness of many WDS problems, researchers have moved away from deterministic methods toward heuristic optimization techniques (Savić et al. 2018), which typically are coupled with hydraulic solvers such as EPANET. WDS optimization uses a variety of metaheuristics, such as genetic algorithms and their variations (Prasad and Park 2004; Savić and Walters 1997), simulated annealing (Cunha and Sousa 1999), or particle swarm optimization (Suribabu and Neelakantan 2006). The principal advantage is that these methods are able to solve challenging problems that no problem-specific deterministic algorithm currently can solve efficiently. For example, Araujo et al. (2006) used EPANET and a genetic algorithm (GA) to optimize the number of valves and their locations for pressure control in the WDS. Bonthuys et al. (2020) developed an optimization procedure for energy recovery and reduction of leakage utilizing a GA with the hydraulic modeling performed in EPANET. Ant colony optimization was used by López-Ibáñez et al. (2008) in conjunction with EPANET for optimal control of pumps in WDS; warnings issued by EPANET for the inefficient operation of pumps were used in the constraint-handling procedure. However, a survey of control optimization for WDS (Mala-Jetmarova et al. 2017) concluded that even with parallel programming techniques and more-efficient deterministic optimization methods, WDS simulations still may be computationally prohibitive for real-time control.

Both deterministic and stochastic optimization methods grapple with challenges in real-time control scenarios, in which the optimal set of actions must be determined based on continuous measurements collected in real time. Consequently, a trade-off between method efficiency and precision must be struck, resulting in simplified hydraulic models and/or a very limited computing budget for the optimization procedure, which has an impact on the solution quality. In this paper we show that the use of deep reinforcement learning in WDS optimization can alleviate these limitations.

In recent years, DRL has revolutionized sequential decision-making, achieving ground-breaking results in several fields, including superhuman performance in chess (Silver et al. 2018) and Go (Silver et al. 2016), protein folding prediction (Jumper et al. 2021), control of traffic lights (Wiering et al. 2000), autonomous driving (Kiran et al. 2022), and robotic control (Kalashnikov et al. 2018). For stormwater systems, Mullapudi et al. (2020) applied reinforcement learning for real-time control using the Deep Q-Network algorithm, and limited the action space to only 27 possible actions. In WDS, Lee and Labadie (2007) used reinforcement learning for stochastic optimization of multireservoir systems; Hajgató et al. (2020) used DRL for real-time optimization of pumps in WDS, and found that their agent is capable of performing as well as the best conventional techniques but is 2 times faster. They also noted the advantage of DRL for real-time control compared with previous methods. Mosetlthe et al. (2020) used DRL with a quadratic approximation of WDS hydraulics to predict the optimal pressure distribution by controlling pressure reducing valves (PRVs). Their emphasis was on the model-free nature of their approach, and their treatment of the DRL method used was minimal. To the best of our knowledge, these papers and others that applied DRL for WDS were restricted to small action spaces, on the order of dozens of possible actions at most.

In DRL, an optimal control policy is learned from the experience collected by the dynamic interaction with the environment, which in this paper was approximated by a WDS model simulated in EPANET. DRL has numerous benefits that can be exploited for optimal control in WDS, in particular, scaling to high-dimensional problems and dealing with stochastic variables (water demands,

energy costs and rates, and so forth). In addition, AI-based methods such as DRL are extremely suitable for real-time control problems because only policy inference is required after training, and continuous learning is possible after deployment.

We present a technique for optimizing pressure control in WDS using the widely used hydraulic software EPANET version 2.2, introducing a heuristic approach that integrates this solver with DRL. Hajgató et al. (2020) proved that this approach can improve pump performance in real time. Here, we use it for pressure control, and highlight its generality for other WDS optimal control problems for large action spaces and in a stochastic context.

To deal with high-dimensional action spaces, we use a reinforcement learning algorithm based on a branching neural network architecture (BDQ). This algorithm is scalable to large action spaces and can be used for discrete action values such as open or closed valves and on or off pumps, as well as continuous actions and mixed action spaces.

We demonstrate the superiority of the BDQ algorithm over the classical Deep Q Network (DQN) on high-dimensional action spaces. Then, we modify the BDQ algorithm to deal with random pipe break scenarios. We present BDQ with fixed actions (BDQF), which is superior to BDQ in this situation, particularly when the allowable actions at each time step are scarce. To the best of our knowledge, this methodology is the first to deal with pressure control in the event of isolating some sectors due to random pipe bursts. Moreover, this framework is sufficiently general to address a wide variety of sequential decision optimization problems in WDS, and can be used for real-time control in intelligent WDS.

We used Brockman et al.'s (2016) optimized OpenAI Gym interface of the EPANET solver in Python, as well as our own implementation of the BDQ algorithm, which we contributed to Tianshou (Weng et al. 2021), which is an open-source library for deep reinforcement learning licensed under the MIT license.

The remainder of this paper is organized as follows. Section "Basic Principles of Reinforcement Learning" provides an overview of the fundamental concepts of reinforcement learning, discussing policies and value functions, as well as the policy iteration technique. Next, section "Framing the WDS Pressure Control as a RL Problem" frames the WDS pressure control problem as a reinforcement learning problem, detailing the states, actions, and rewards. Section "DRL Algorithms Used" introduces the deep reinforcement learning algorithms employed in this research, including the DQN and BDQ algorithms. We then present the BDQF algorithm, which is an adaptation of BDQ specifically designed to address the pipe failure scenario. Section "Results" presents the findings of the study, which demonstrate the effectiveness of the proposed methodology in optimizing pressure control under normal conditions and in the presence of random pipe burst incidents. Lastly, section "Conclusion and Future Work" concludes the paper and provides directions for future work in this area.

Basic Principles of Reinforcement Learning

The problem of reinforcement learning can be formalized using ideas from dynamical systems theory, specifically, as the optimal control of incompletely known Markov decision processes (MDPs).

We briefly review the main elements of the reinforcement learning framework necessary to present the algorithms used in this work. A detailed and rigorous introduction was presented by Sutton and Barto (2018).

In the general setting presented in Fig. 1, the agent is a learner and decision maker. The thing with which it interacts, comprising

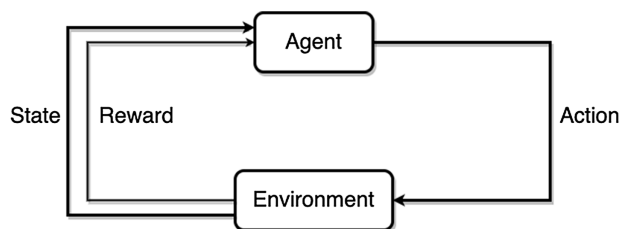


Fig. 1. Agent–environment interaction in a Markov decision process.

everything outside the agent, is called the environment. At each time-step t , the agent receives some representation of the state S_t of the environment and has to take an action A_t . Following that, the agent receives a reward R_t , and the environment transitions to a new state S_{t+1} . The state transitions and rewards are stochastic and are assumed to have the Markov property; i.e., they depend only on the immediate state of the environment S_t and action A_t taken by the agent.

In reinforcement learning, the agent will be learning, by interacting with the environment, to take the actions that maximize the expected discounted total reward it obtains $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R_t]$, where $\gamma \in [0, 1)$ is a factor discounting future rewards. This can be achieved by (1) learning directly to choose the best action for a given state, a class of algorithms that is called policy-based algorithms; or (2) learning indirectly, by learning a value function, and selecting the action with the highest value for that state, a class of algorithms that is called value-based algorithms. This paper used two value-based algorithms.

Policies and Value Functions

Value functions are state functions (or state–action pair functions) that estimate the future rewards that can be expected in a particular state (or in a particular state–action pair). The rewards that the agent can expect to receive in the future depend on the actions it takes. Accordingly, value functions are defined with respect to particular ways of acting, called policies.

Formally, a policy is a mapping from states to probabilities of selecting each possible action. If the agent is following policy π at time t , then $\pi(a|s)$ is the probability that $A_t = a$ if $S_t = s$.

The value function of a state s under a policy π , denoted $V_{\pi}(s)$, is the expected reward starting in s and following π thereafter

$$V_{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=t}^{\infty} \gamma^k R_k | s \right] \quad (1)$$

Similarly, the value of taking action a in state s under a policy π , denoted $Q_{\pi}(s, a)$ and called the q -value function, is defined as the expected return starting from s , taking the action a , and thereafter following policy π

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=t}^{\infty} \gamma^k R_k | s, a \right] \quad (2)$$

A third value function, $A_{\pi}(s, a)$, called the advantage function, measures how advantageous it is to take action a compared with the expected reward in state s , following the policy π

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s) \quad (3)$$

Policy Iteration

Policy iteration (Sutton and Barto 2018) is a method for solving a Markov decision process by iteratively improving an initial policy. It involves alternating between two main steps: policy evaluation, which estimates the value of each state under the current policy; and policy improvement, which updates the policy based on the updated values. The process continues until the policy converges to the optimal solution or a satisfactory solution is found. In temporal difference (TD) learning, policy evaluation consists of estimating the value function of the current policy π by updating $Q_{\pi}(s, a)$ in the direction of

$$r + \gamma Q_{\pi}(s', \pi(a|s')) \quad (4)$$

The policy improvement step updates the policy so that for each state s , the new policy π' selects the action a that maximizes the action value function. This can be written

$$\pi'(a|s) = \underset{a}{\operatorname{argmax}} Q_{\pi}(s', a) \quad (5)$$

Framing the WDS Pressure Control as a RL Problem

Water distribution networks can be modeled as a collection of links connecting nodes. Water flows along links and enters or leaves the system at nodes. All the actual physical components of a distribution system can be represented in terms of these constructs. One particular scheme for accomplishing this is shown in Fig. 2, in which links consist of pipes, pumps, or control valves. Pipes convey water from one point to another, pumps raise the hydraulic head of water, and control valves maintain specific pressure or flow conditions. Nodes consist of pipe junctions, reservoirs, and tanks. Junctions are demand nodes at which links connect and at which water consumption occurs. Reservoir nodes represent fixed-head boundaries, such as lakes, groundwater aquifers, treatment plant clear wells, or connections to parts of a system that are not being modeled. Tanks are storage facilities, the volume and water level of which can change over an extended period of system operation.

Our objective was to train a reinforcement learning agent to control valves in the WDS to minimize the overall pressure in the network under the constraints of minimum and maximum pressure, and without emptying or overfilling the tanks.

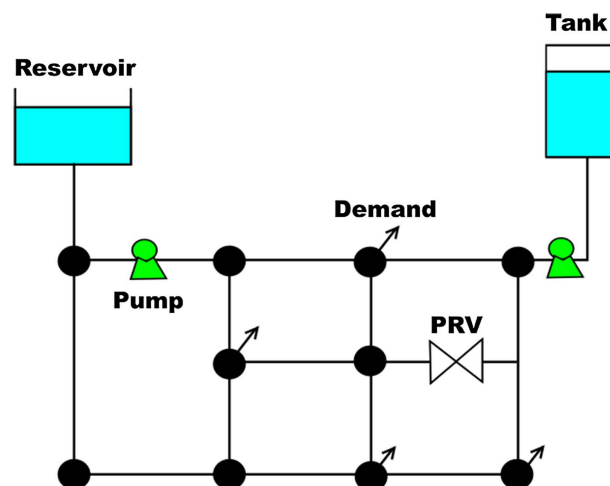


Fig. 2. Node–link representation of a WDS.

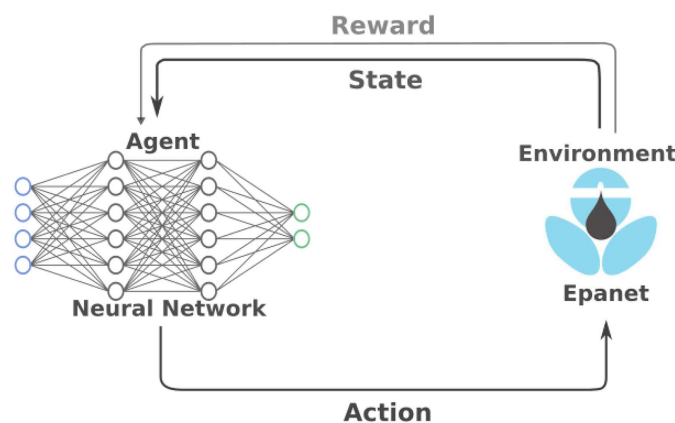


Fig. 3. WDS reinforcement learning setup.

We use a time-step of 1 h, which is the usual time step for hydraulic simulations in WDS. We consider a full-length episode to be 24 h, which corresponds to the usual cycle of WDS's operations. At each time step, the agent receives the pressures at demand nodes, the tank levels, and the time; this is the state s of the environment. The agent then should determine the opening values of the valves; these are the actions a . A simulation then is performed via EPANET to calculate the new pressures at the demand nodes and the new tank levels; the environment transitions to the new state s' . The reward r then is calculated and returned to the agent. The quadruple $e = (s, a, r, s')$ is called an experience. Fig. 3 provides a schematic illustration of the process.

States

The states are the part of the environment that are relevant to the agent—they are what the agent takes as input. For our WDS environments, the states are composed of the pressures at the demand nodes, the levels at the tanks, and the current time. Therefore, for a WDS with N demand nodes and T tanks, the state space is a continuous space with a dimension size equal to $N + T + 1$.

Actions

The actions are opening and closing the control valves for discrete actions, and setting the values of the valves in the case of continuous actions.

Rewards

Rewards depend on two terms:

- **Pressures:** Our goal is to ensure service for the highest number of clients while maintaining operational constraints. Hence, we use a cost function c_p for counting the nodes that are served with pressures within the required range. The value of c_p is equal to a number between one and two for nodes with pressures between $p_{\min}$ and $p_{\max}$, and to zero otherwise; it is biased toward smaller pressures to incentivize pressure reduction (Fig. 4). These values are summed and normalized over the number of nodes.
- **Tanks:** The majority of water-distribution networks operate on a 24-h cycle in which the storage tanks are refilled overnight when the charge for electricity is low, and then are drawn down during the daytime hours when demands are high. This not only reduces the operating costs but also ensures a turnover of the water in storage, thereby avoiding stagnation. It is very important to

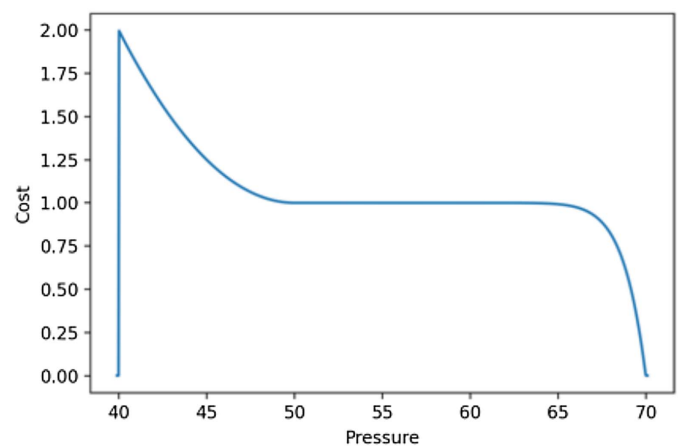


Fig. 4. Cost function with smooth transitions.

avoid overtopping or emptying the tanks; these events are considered to be episode-ending incidents.

Therefore the reward at time-step t is formalized as

$$r_t = \sum_{i=1}^N \frac{c_p(p_i)}{n}$$

where p_i = pressure at demand node i ; and N = number of nodes in the WDS.

The goal of the RL agent is to maximize the cumulative discounted reward over the episode

$$G = \sum_{t=0}^{t_f} \gamma^t r_t$$

where t_f = 23 for a noninterrupted episode, and otherwise is the time at which the incident of overtopping or emptying any of the tanks occurs.

DRL Algorithms Used

DQN and BDQ

The DQN is a classical value-based deep reinforcement learning algorithm introduced by Deepmind in 2015 (Mnih et al. 2015), igniting the current RL revolution, and has been used since then in numerous publications. However, the DQN suffers from the curse of dimensionality, as the network representing the Q-values scales exponentially with respect to the number of dimensions in the action space. The BDQ (Tavakoli et al. 2017) was introduced to solve this problem by adopting a branching architecture that represents each dimension in the action space as a branch, and has a common network trunk that coordinates between the branches.

BDQF: Adapting BDQ for the Pipe Failure Scenario

If a pipe bursts in a specific network sector, some PRVs must be turned off to isolate the affected sector. This means that the agent no longer can change the status of these valves, and some actions therefore are unavailable. Previous research (Vinyals et al. 2017; Berner et al. 2019; Ye et al. 2019) used action masking to address this issue in policy gradient algorithms, which involves keeping the action space fixed and considering only valid actions at a given state. However, for value-based algorithms such as the DQN and

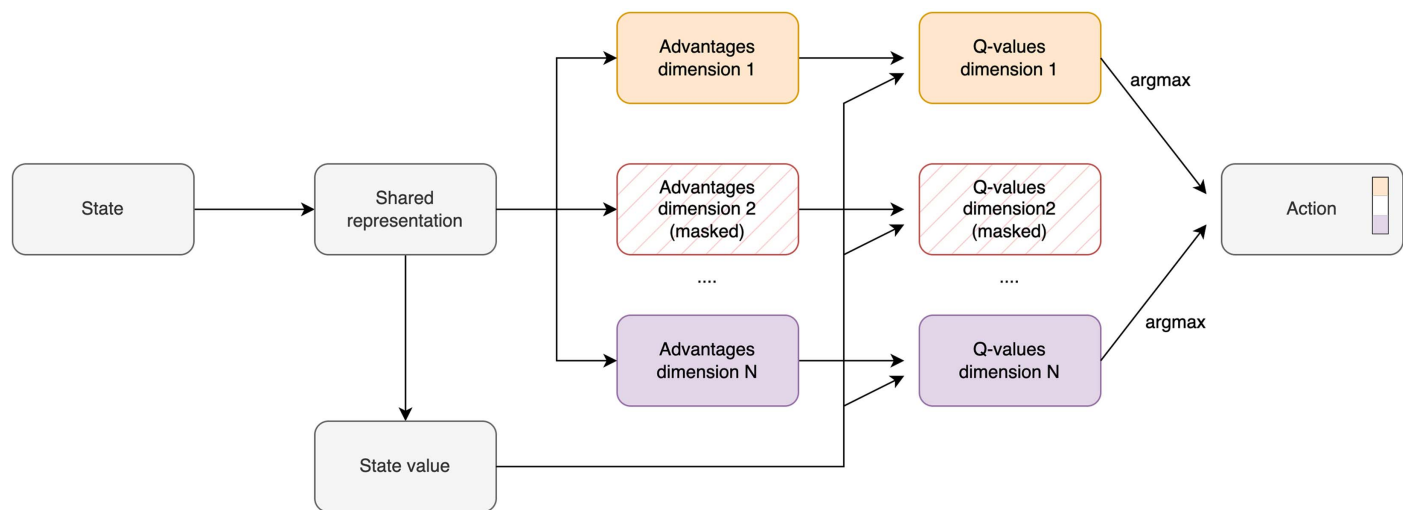


Fig. 5. Action branching network used in the BDQF algorithm. Fixed actions dimensions are masked to prevent gradient propagation.

BDQ, we were unable to find any papers about using action masking. The only mentions of action masking with value-based algorithms that we found were in a blog post (Zouitine 2021) and in the implementation of Tianshou's DQN algorithm, but without any references. In both cases the idea consisted of assigning low or negative Q-values to masked actions to prevent them from being selected by the policy $\pi(a|s) = \text{argmax}_a Q(s, a)$. We propose a different method for action masking with value-based algorithms, which involves using the branching architecture of the BDQ to avoid attributing rewards to fixed actions.

First, we observe that fixing some actions in the action space and using a vanilla RL algorithm still results in a valid policy; however, we show theoretically that such an algorithm is inefficient, and we propose BDQF as a better solution, which we validated using the experimental results.

For a given pipe failure, let V_f represent the subset of valves that must be closed, A_f represent the elements of V_f that are in the action space, and A_m represent the remainder of the action space. The agent's actions can be represented as $a = (a_m, a_f)$, where $a_m \in A_m$ are the manipulated actions, and $a_f \in A_f$ are the fixed actions. The crucial point is that when action a is performed, only the a_m portion has effects on the environment (because a_f are fixed, and their corresponding valves will remain closed even if the policy attempts to assign them different values). Consequently, $r(s, a) = r(s, a_m)$, and $Q(s, a) = Q(s, a_m)$ follows, indicating that learning $Q(s, a)$ is equivalent to learning $Q(s, a_m)$; therefore, any RL algorithm can generate a valid policy despite fixing some actions in the action space. However, learning $Q(s, a)$ is less efficient and contains unnecessary duplicates: $Q(s, (a_m, a_f))$ is the same as $Q(s, (a_m, a'_f))$ for any (a_f, a'_f) , and for better algorithm learning only $Q(s, a_m)$ should be sufficient. Moreover, in the case of function approximation, this inefficiency will manifest as a learning noise corresponding to the assignment of rewards to ineffectual actions.

Leveraging the benefit of separating the action dimensions in BDQ can mitigate these issues. We propose masking the action dimensions with fixed values by preventing the back-propagation of the learning signal in action-fixed branches. This will allow the correct Q values to be learned without artificially assigning low or negative values. We modify the temporal-difference target and the loss function of the original BDQ algorithm (Tavakoli et al. 2017) as follows:

$$y = r + \gamma \frac{1}{N_m} \sum_{d_m} Q'_d(s, \text{argmax}_{a' \in A_{d_m}} Q_\theta(s', a')) \quad (6)$$

$$L = \mathbb{E}_{(s, a, r, s') \sim D} \left[\sum_{d_m} (y_{d_m} - Q_{d_m}(s, a_{d_m}))^2 \right] \quad (7)$$

where N_m and d_m = number and dimensions of the manipulated part of the action space, respectively. Only the manipulated actions dimensions are considered for selecting the best actions and for updating the Q-values. Fig. 5 is a schematic representation of the BDQF algorithm. The detailed algorithm is presented in Appendix I.

Results

Optimizing Pressure Control in Normal Conditions

We applied the DQN and BDQ algorithms to a set of nine real WDS. Figs. S1–S10 present the layouts of these networks, and Table 1 presents an overview of their main characteristics. We operated under normal conditions, with no pipe-bursting incidents. These networks contain either two or four control valves, each of which can be set to any value between 10 and 50 pressure control actuation (PCA). The action space is discretized by dividing the interval [10, 50] into eight equal bins; this value corresponds to a 5-PCA step, which was determined to be an adequate control step based on the input from experts in the field of water distribution systems. By employing a control step of this size, a balance between the granularity of control actions and the complexity of the problem is achieved, allowing for effective and efficient optimization.

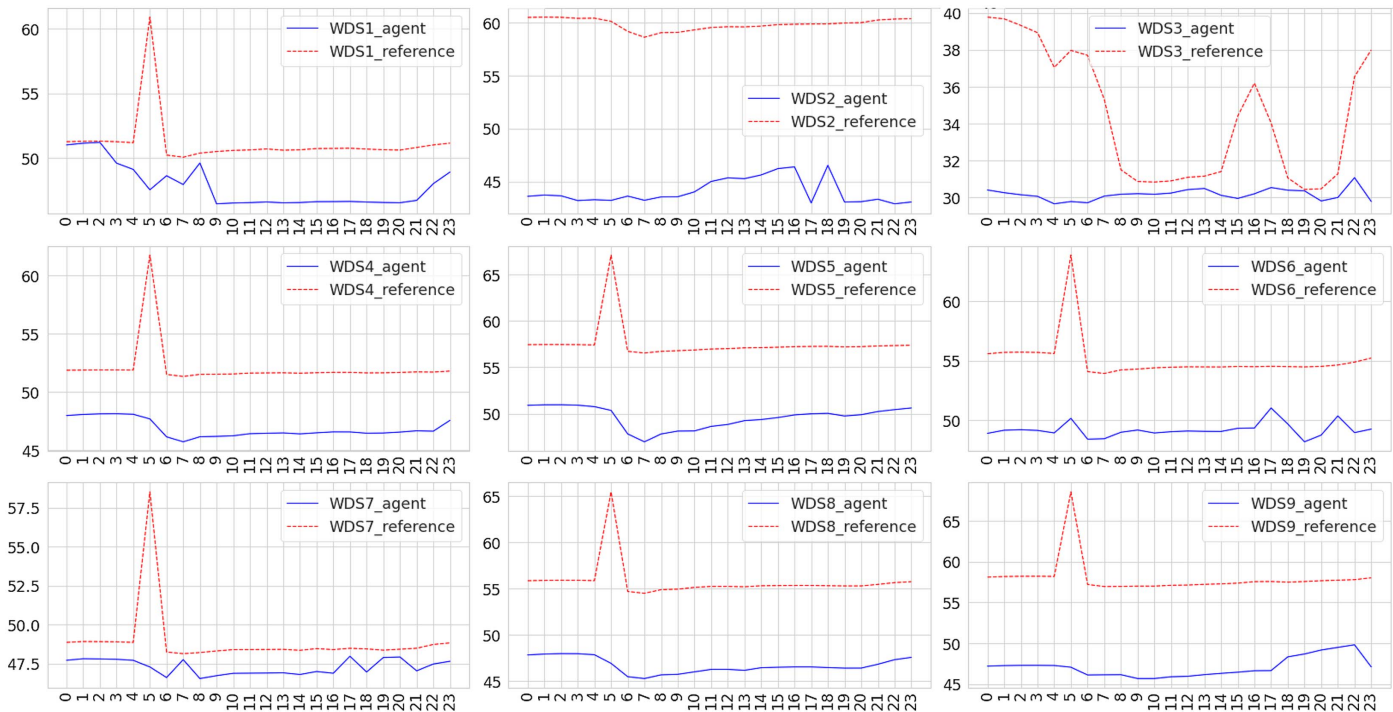
In every WDS, the RL agents were able to find better solutions than the reference used by operators, achieving an improvement of as much as 26% in mean pressure, with an average of 13% improvement across all WDS; Fig. 6 presents the improvement in pressure for each network.

Fig. 7 summarizes the learning curves of the BDQ and DQN across all nine WDS. Each of the 25 randomly seeded runs that were executed required 48 h to complete. We normalized the rewards and calculated bootstrapped confidence intervals in accordance with

Table 1. Mean pressures before and after optimization using Deep RL

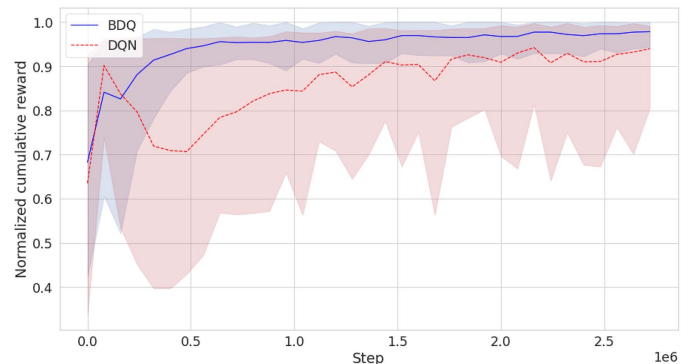
WDS name	Nodes	Edges	Valves	$ A $	Before	After	Improvement (%)
WDS1	2,418	2,552	4	256	34.4	30.1	12
WDS2	3,153	3,274	4	256	55.1	49.2	11
WDS3	1,841	1,900	2	16	48.9	47.3	3
WDS4	2,756	2,869	2	16	52.1	46.8	10
WDS5	1,328	1,373	2	16	58	47	19
WDS6	1,161	1,194	2	16	57.6	49.6	14
WDS7	2,275	2,364	2	16	51.2	47.9	7
WDS8	2,370	2,463	2	16	55.8	46.7	16
WDS9	1,902	1,964	2	16	59.9	44	26

Note: Cases in which BDQ achieved better rewards than DQN are indicated in bold.

**Fig. 6.** Mean pressures of the reference solution and the best RL agent solutions.

Agarwal et al. (2021). Although the best rewards at the test time were very similar, the BDQ learned more quickly and was more stable than the DQN (Fig. 7).

Table 1 includes a description of each network and the average pressures before and after optimization with Deep RL. Text in bold indicates instances in which the BDQ achieved greater rewards than the DQN. In all instances, the best cumulative reward achieved by both algorithms was nearly identical, differing by no more than 0.5%; this was expected because the action space for these networks was relatively small, with a maximum of $8^4 = 256$ possible actions. However, when 32 bins are considered in each action dimension for four valves, the resulting action space size is $32^4 = 1,048,576$. In this scenario, the DQN performed significantly worse than the BDQ (Fig. 8), and if the number of bins was increased to 50, for example, the 16 GB Nvidia Tesla v100 GPU that we used no longer could fit the DQN network in its memory. Due to the manner in which DQN networks represent the Q-values, with one neuron in the output layer for each possible action, the network scales exponentially with the number of dimensions of action space. In contrast, the BDQ employs a branching architecture that permits it to scale linearly with the number of action space dimensions. This

**Fig. 7.** Performance of BDQ and DQN on WDS1–WDS9 with bootstrapped 95% confidence interval (CI).

results in a sevenfold improvement in performance for the BDQ over the DQN in the case of 32 bins, for which the BDQ can execute 7 million steps while DQN can only execute 1 million with the same computation budget (Fig. 8). As more control elements

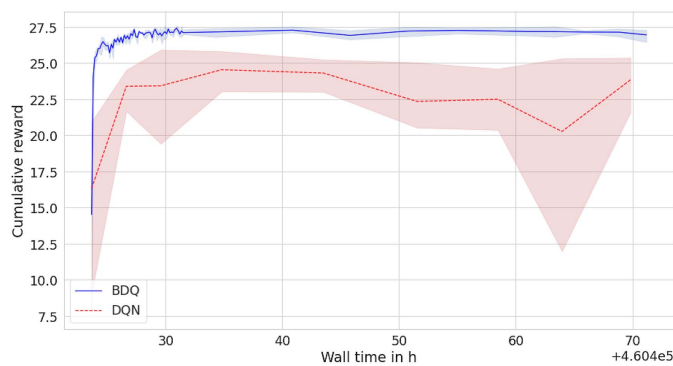


Fig. 8. Performance of BDQ and DQN on WDS4 with 32 bins for each of the 4 valves.

are added to the WDS, the performance gap increases exponentially, meaning that only the BDQ can be used for more-complex cases.

Optimizing Pressure Control in the Presence of Random Pipe Bursts Incidents

We consider the scenario of random leaks in the network. The WDS considered in this case has 2,565 nodes, 2,936 links, and 18 open or closed control valves that the agent can manipulate to stabilize the pressure. The WDS is composed of a transmission system of large pipes that transports the water to the distribution system that distributes the water to the demand nodes. The water distribution system is divided into 10 sectors; this division is achieved by considering the transmission system (i.e., the part of the network that transports higher water flows along large distances within the WDS). By removing this transmission system from the network, the remaining connected nodes in the graph define the individual sectors. Each of the 10 sectors can be isolated if a pipe bursts in one of them. Some control valves must remain closed when isolating a sector and cannot be changed by the agent, which represent a challenge to vanilla RL algorithms because they assume a fixed action space in which all actions are available at each time-step. We used the BDQ without any modifications with some drawbacks, we also used our modified version that takes into account fixed actions (i.e., the BDQF). We trained the two agents for 20 randomly seeded runs of 48 h each. During training, each episode was started by a random pipe burst that corresponded to isolating one of the 10 sectors. The agents were evaluated for each of the 10 possible pipe burst sectors, and the mean reward was calculated over these evaluations.

In this case, there is no reference solution against which the performance of the agents can be compared. However, there was a clear learning tendency for both algorithms, because both the BDQ and the BDQF were successful in learning good policies that prevented the premature termination of episodes due to emptying or overfilling of the tanks, and both achieved high rewards. The BDQF's best runs yielded a 10-sector average reward of 20.2, whereas the BDQ's best run yielded a 10-sector average reward of 19.3. To compare the performance of both algorithms, we plot the bootstrapped 95% confidence interval (CI) of the 10 sectors' mean reward over 20 runs (Fig. 9); the BDQF outperformed the BDQ. This is more evident in Fig. 10, for which we modified the action space to have sparse free actions that are controlled by the agent (we added 100 dummy valves that appeared in the action space but that had no effect on the environment).

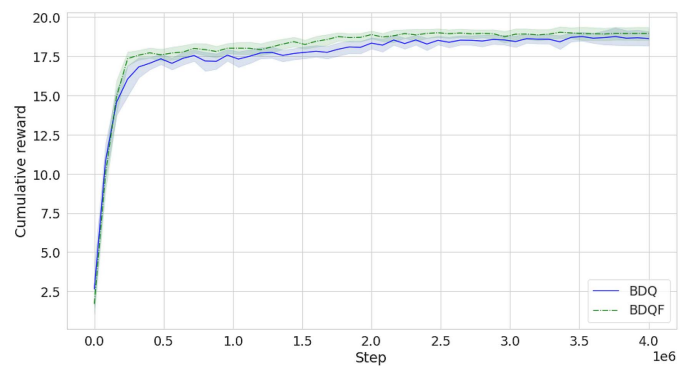


Fig. 9. Performance of BDQ and BDQF for pressure control in the presence of random leaks.

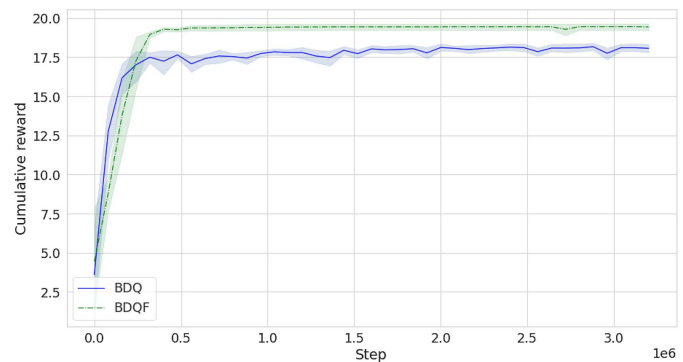


Fig. 10. Performance of BDQ and BDQF for pressure control in the presence of random leaks with sparse free actions.

Conclusion and Future Work

This work

- introduces DRL-EPANET, a method for optimal pressure control in water distribution systems using deep reinforcement learning and EPANET;
- demonstrated the effectiveness of the approach on 10 real-world WDS, improving mean pressure by as much as 26%;
- demonstrated the advantage of using the BDQ rather than the DQN for large action spaces;
- applied the BDQ for optimal pressure control in the presence of random pipe bursts; and
- developed the BDQF, which is an improvement of the BDQ, to mitigate learning noise from actions that have no environmental impact.

The DRL-EPANET framework can be used to tackle a wide range of sequential decision optimization problems in WDS, such as pressure control as we demonstrated in this paper, pump optimization, energy, water quality, and so forth, or any combination of the aforementioned problems. We believe that any WDS sequential decision problem that can be simulated using EPANET can, in principle, be solved in this framework. Moreover, DRL-EPANET provides a real-time control solution, because after it is trained, the agent is capable of reacting in real time to changes in the network, with only inference calculation needed, usually on the order of milliseconds, i.e., without recalculating the solution, which can be a limiting factor for other methods. This paper demonstrates that DRL-EPANET can deal with the stochasticity of the WDS

environment in the form of random pipe bursts. Similarly, the DRL-EPANET agent can be trained with more stochastic scenarios to deal with demand uncertainty, contamination, and other component failures scenarios.

In the future, we would like to use model-based DRL for better sample efficiency. Previous work has shown that artificial neural networks can be used as a substitute for EPANET simulation of WDS with a high degree of accuracy (Rao and Alvarruiz 2007), which indicates that model-based DRL can be expected to work quite well for WDS. Another direction for research is using graph neural networks, which are the most natural representation for a WDS and can constitute a strong induction bias for DRL that might help in learning more efficiently and lead to better solutions. Hajgató et al. (2021) have shown that such an approach works for supervised learning for reconstructing nodal pressure in WDS. We also hope to quantify the uncertainty of the agent actions in production, so that we can defer control to the operator in uncharted territory if the agent is uncertain.

Appendix I. Computer Resources

The runs were performed on the CTE-power9 cluster of the Barcelona Supercomputing Center. Each run used one Tesla V100 GPU and 40 CPUs to run 24 EPANET environments in parallel as well as the learning algorithm. There were a maximum of 10 runs at the same time. The run-time was about 48 h.

Appendix II. BDQF Algorithm

Algorithm 1. BDQ for fixed actions (BDQF)

- 1 **Hyperparameters:** replay buffer capacity M , reward discount factor γ , delayed steps C for target network update, greedy factor ϵ , and number of steps T for testing the model
- 2 **Inputs:** number of episodes N , environment Env takes in the current state–action pair and outputs the reward and next state
- 3 Initialize parameters θ of action-value function Q
- 4 Initialize target network Q' with parameters $\theta' \leftarrow \theta$
- 5 Fill the replay buffer $\mathcal{D}$
- 6 **For** episode = 0, 1, 2, . . . , N **do**
- 7 Initialize environment with leak, get fixed actions a_f and initial state s_0
- 8 **For** Step $t = 0, 1, 2, \dots$ **do**
- 9 With probability ϵ select a random action vector a_t ; otherwise select $a_t = (\arg\max_{a_d \in A_d} Q_\theta(s_t, a_d))$
- 10 $r_t, s_{t+1} \leftarrow \text{Env}(s_t, a_t)$
- 11 If the episode has ended, set $f_t = 1$. Otherwise set $f_t = 0$
- 12 Store experience $E = (a_t, a_t, r_t, f_t, s_{t+1})$ in $\mathcal{D}$ FIFO
- 13 Sample prioritized minibatch of transitions $E = (s_i, a_i, r_i, f_i, s_{i+1})$ from $\mathcal{D}$
- 14 If $f_i = 0$, set the common target $y_i = r_i + \gamma(1/N_m) \times \sum_{d_m} Q'_d(s, \arg\max_{a' \in A_{d_m}} Q_\theta(s', a'))$. Otherwise, set $y_i = r_i$
- 15 Perform a gradient descent step on $\sum_{d_m} (y_{i_{d_m}} - Q_{d_m}(s_i, a_{i_{d_m}}))^2$
- 16 Synchronize the target Q' every C steps
- 17 Evaluate and save the model every T steps
- 18 If the episode has ended, break the loop
- 19 **End for**
- 20 **End for**

Appendix III. Hyperparameters of DQN

Hyperparameter	Value
Hidden-sizes	[512, 256]
Epsilon-train	0.73
Epsilon-test	0.01
Epsilon-decay	5×10^{-6}
Buffer-size	10^5
Learning-rate	8×10^{-5}
Gamma	0.99
Target-update-freq	500
Epoch	1,000
Step-per-epoch	8×10^4
Step-per-collect	24
Update-per-step	1/24
Batch-size	128
Training-envs-number	24
Test-envs-number	1

Appendix IV. Hyperparameters of BDQ and BDQF

Hyperparameter	Value
Common-hidden-sizes	[512, 256]
Action-hidden-sizes	128
Value-hidden-sizes	128
Epsilon-train	0.73
Epsilon-test	0.01
Epsilon-decay	5×10^{-6}
Buffer-size	10^5
Learning-rate	8×10^{-5}
Gamma	0.99
Target-update-freq	500
Epoch	1,000
Step-per-epoch	8×10^4
Step-per-collect	24
Update-per-step	1/24
Batch-size	128
Training-envs-number	24
Test-envs-number	1

Data Availability Statement

Our implementation of the BDQ algorithm has been made available as a part of the open-source Tianshou library (<https://github.com/thu-ml/tianshou>) under the name Branching DQN. The rest of the code and the WDS data are proprietary and sensitive data that belong to Aigües de Barcelona, and may be provided only with restrictions.

Supplemental Materials

Figs. S1–S10 appear online in the ASCE Library (www.ascelibrary.org).

References

- Agarwal, R., M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare. 2021. “Deep reinforcement learning at the edge of the statistical precipice.” *Adv. Neural Inf. Process. Syst.* 34 (Dec): 29304–29320. <https://doi.org/10.48550/arXiv.2108.13264>.
- Araujo, L., H. Ramos, and S. Coelho. 2006. “Pressure control for leakage minimisation in water distribution systems management.” *Water*

- Resour. Manage.* 20 (1): 133–149. <https://doi.org/10.1007/s11269-006-4635-3>.
- Berner, C., et al. 2019. “Dota 2 with large scale deep reinforcement learning.” Preprint, submitted December 13, 2019. <http://arxiv.org/abs/1912.06680>.
- Bonthuys, G. J., M. van Dijk, and G. Cavazzini. 2020. “Energy recovery and leakage-reduction optimization of water distribution systems using hydro turbines.” *J. Water Resour. Plann. Manage.* 146 (5): 04020026. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0001203](https://doi.org/10.1061/(ASCE)WR.1943-5452.0001203).
- Brockman, G., V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. 2016. “OpenAI Gym.” Preprint, submitted June 5, 2016. <http://arxiv.org/abs/1606.01540>.
- Cunha, M. D. C., and J. Sousa. 1999. “Water distribution network design optimization: Simulated annealing approach.” *J. Water Resour. Plann. Manage.* 125 (4): 215–221. [https://doi.org/10.1061/\(ASCE\)0733-9496\(1999\)125:4\(215\)](https://doi.org/10.1061/(ASCE)0733-9496(1999)125:4(215)).
- Deuerlein, J. W., A. R. Simpson, and S. Dempe. 2009. “Modeling the behavior of flow regulating devices in water distribution systems using constrained nonlinear programming.” *J. Hydraul. Eng.* 135 (11): 970–982. [https://doi.org/10.1061/\(ASCE\)HY.1943-7900.0000108](https://doi.org/10.1061/(ASCE)HY.1943-7900.0000108).
- Hajgató, G., B. Gyires-Tóth, and G. Paál. 2021. “Reconstructing nodal pressures in water distribution systems with graph neural networks.” Preprint, submitted April 28, 2021. <http://arxiv.org/abs/2104.13619>.
- Hajgató, G., G. Paál, and B. Gyires-Tóth. 2020. “Deep reinforcement learning for real-time optimization of pumps in water distribution systems.” *J. Water Resour. Plann. Manage.* 146 (11): 04020079. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0001287](https://doi.org/10.1061/(ASCE)WR.1943-5452.0001287).
- Jumper, J., et al. 2021. “Highly accurate protein structure prediction with AlphaFold.” *Nature* 596 (7873): 583–589. <https://doi.org/10.1038/s41586-021-03819-2>.
- Kalashnikov, D., et al. 2018. “Scalable deep reinforcement learning for vision-based robotic manipulation.” In Vol. 87 of *Proc., Conf. on Robot Learning, PMLR*, 651–673. Breckenridge, CO: Proceedings of Machine Learning Research.
- Kiran, B. R., I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez. 2022. “Deep reinforcement learning for autonomous driving: A survey.” *IEEE Trans. Intell. Transp. Syst.* 23 (6): 4909–4926. <https://doi.org/10.1109/TITS.2021.3054625>.
- Lee, J.-H., and J. W. Labadie. 2007. “Stochastic optimization of multireservoir systems via reinforcement learning.” *Water Resour. Res.* 43 (11). <https://doi.org/10.1029/2006WR005627>.
- López-Ibáñez, M., T. D. Prasad, and B. Paechter. 2008. “Ant colony optimization for optimal control of pumps in water distribution networks.” *J. Water Resour. Plann. Manage.* 134 (4): 337–346. [https://doi.org/10.1061/\(ASCE\)0733-9496\(2008\)134:4\(337\)](https://doi.org/10.1061/(ASCE)0733-9496(2008)134:4(337)).
- Mala-Jetmarova, H., N. Sultanova, and D. Savic. 2017. “Lost in optimisation of water distribution systems? A literature review of system operation.” *Environ. Modell. Software* 93 (Jul): 209–254. <https://doi.org/10.1016/j.envsoft.2017.02.009>.
- Mnih, V., et al. 2015. “Human-level control through deep reinforcement learning.” *Nature* 518 (7540): 529–533. <https://doi.org/10.1038/nature14236>.
- Mosetlthe, T. C., Y. Hamam, S. Du, E. Monacelli, and A. A. Yusuff. 2020. “Towards model-free pressure control in water distribution networks.” *Water* 12 (10): 2697. <https://doi.org/10.3390/w12102697>.
- Mullapudi, A., M. J. Lewis, C. L. Gruden, and B. Kerkez. 2020. “Deep reinforcement learning for the real time control of stormwater systems.” *Adv. Water Resour.* 140 (Jun): 103600. <https://doi.org/10.1016/j.advwatres.2020.103600>.
- Prasad, T. D., and N.-S. Park. 2004. “Multiobjective genetic algorithms for design of water distribution networks.” *J. Water Resour. Plann. Manage.* 130 (1): 73–82. [https://doi.org/10.1061/\(ASCE\)0733-9496\(2004\)130:1\(73\)](https://doi.org/10.1061/(ASCE)0733-9496(2004)130:1(73)).
- Rao, Z., and F. Alvarruiz. 2007. “Use of an artificial neural network to capture the domain knowledge of a conventional hydraulic simulation model.” *J. Hydroinf.* 9 (1): 15–24. <https://doi.org/10.2166/hydro.2006.014>.
- Savic, D. A., and G. A. Walters. 1997. “Genetic algorithms for least-cost design of water distribution networks.” *J. Water Resour. Plann. Manage.* 123 (2): 67–77. [https://doi.org/10.1061/\(ASCE\)0733-9496\(1997\)123:2\(67\)](https://doi.org/10.1061/(ASCE)0733-9496(1997)123:2(67)).
- Savić, D., H. Mala-Jetmarova, and N. Sultanova. 2018. “History of optimization in water distribution system analysis.” In Vol. 1 of *Proc., WDSA/CCWI Joint Conf.* Kingston, Canada: Queen’s Univ.
- Schaake, J. C., Jr., and D. Lai. 1969. *Linear programming and dynamic programming application to water distribution network design*. Cambridge, MA: MIT Hydrodynamics Laboratory.
- Silver, D., et al. 2016. “Mastering the game of Go with deep neural networks and tree search.” *Nature* 529 (7587): 484–489. <https://doi.org/10.1038/nature16961>.
- Silver, D., et al. 2018. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play.” *Science* 362 (6419): 1140–1144. <https://doi.org/10.1126/science.aar6404>.
- Suribabu, C., and T. Neelakantan. 2006. “Design of water distribution networks using particle swarm optimization.” *Urban Water J.* 3 (2): 111–120. <https://doi.org/10.1080/15730620600855928>.
- Sutton, R. S., and A. G. Barto. 2018. *Reinforcement learning: An introduction*. Cambridge, MA: MIT Press.
- Tavakoli, A., F. Pardo, and P. Kormushev. 2017. “Action branching architectures for deep reinforcement learning.” Preprint, submitted April 29, 2018. <http://arxiv.org/abs/1711.08946>.
- Vinyals, O., et al. 2017. “StarCraft II: A new challenge for reinforcement learning.” Preprint, submitted August 16, 2017. <http://arxiv.org/abs/1708.04782>.
- Weng, J., H. Chen, D. Yan, K. You, A. Duburcq, M. Zhang, Y. Su, H. Su, and J. Zhu. 2021. “Tianshou: A highly modularized deep reinforcement learning library.” Preprint, submitted January 1, 2022. <http://arxiv.org/abs/2107.14171>.
- Wiering, M. A., et al. 2000. “Multi-agent reinforcement learning for traffic light control.” In *Proc., Machine Learning: Proc., 17th Int. Conf. (ICML 2000)*, 1151–1158.
- Yakowitz, S. 1982. “Dynamic programming applications in water resources.” *Water Resour. Res.* 18 (4): 673–696. <https://doi.org/10.1029/WR018i004.p00673>.
- Ye, D., et al. 2019. “Mastering complex control in MOBA games with deep reinforcement learning.” Preprint, submitted April 3, 2022. <http://arxiv.org/abs/1912.09729>.
- Zouitine, A. 2021. “Masking in deep reinforcement learning.” Adil Zouitine blog. Accessed June 16, 2022. <https://boring-guy.sh/posts/masking-rl/>.